

SEC 10-K Risk Factor Delta Agent

Technical Report · Multi-Step LLM Agent Assignment

1. Problem Statement

An investor or analyst tracking a company's risk profile must ordinarily read two consecutive annual 10-K filings — each between 50 and 200 pages — locate the Risk Factors section in each, and manually identify what changed, what was added, and what disappeared. For a portfolio of even ten companies, this is a multi-day task prone to anchoring bias: analysts tend to notice changes in risks they were already watching and overlook entirely new concerns disclosed for the first time.

This agent automates that workflow by accepting a single ticker symbol and producing an investor-ready memo that summarises exactly how the company's disclosed risk profile changed between its two most recent annual filings, enriched with current news evidence for every material new or changed risk.

A single prompt cannot accomplish this. The task requires three fundamentally different reasoning operations: extraction (identifying individual atomic risk concerns from dense regulatory prose), semantic comparison (deciding whether two differently-worded passages across two filing years describe the same underlying concern or represent a genuinely new one), and synthesis (writing a coherent, investor-facing narrative from a structured diff with supporting news evidence). Collapsing all three into one prompt consistently produces degenerate results in testing: the model loses track of individual risk identities during comparison, produces false additions and removals for risks that merely received cosmetic rewording, and mixes low-level extraction with high-level synthesis in ways that make both unreliable.

2. Chain Design

Step 1 (LLM — Normalizer) receives the raw user-supplied string and returns a `TickerInfo` object containing a canonical uppercase ticker, the full company name, and a confidence level. This step exists because every downstream step depends on an unambiguous, uppercase ticker: the SEC EDGAR CIK lookup in Step 2 performs an exact-match dictionary lookup and silently returns nothing for a lowercase or misspelled symbol.

Step 2 (Tool — SEC EDGAR Fetcher) uses the normalised ticker to query the SEC's public submissions API, retrieves the two most recent 10-K filings, fetches their HTML, and extracts the Item 1A (Risk Factors) section via regex anchors. This step is a tool call and not an LLM call because the agent cannot hallucinate filing text: it must retrieve the actual document from EDGAR.

Step 3 (LLM — Risk Extractor) makes two sequential LLM calls — one for each year's Item 1A text — and atomises the prose into a typed list of `Risk` objects: one discrete concern per object,

with a stable `snake_case` identifier, a label capped at twelve words, a 2–3 sentence summary, and a verbatim excerpt. This step is separate from Step 4 because extraction and comparison are distinct cognitive operations that degrade when combined. Tested on TSLA's FY2024 filing, a single prompt asked to both extract and compare produced seventeen spurious "added" verdicts for risks that existed in both years under slightly different phrasing; the two-step design reduced spurious additions to zero on the same input.

Step 4 (LLM — Differ) receives the two typed Risk arrays and produces a `RiskDiff`: for every risk in either year, it assigns a verdict (added, removed, materially_changed, or unchanged), pairs risks across years by semantic content rather than string similarity, and writes a one-to-two-sentence rationale for each verdict. Step 4 is separate from Step 3 because semantic comparison across two years requires the risks to already be in atomic, short-label form; giving Step 4 raw regulatory prose would require it to simultaneously parse two years of dense text and reason about cross-year equivalence — a multi-objective task where both objectives suffer.

Step 5 (Tool — News Search) reads the diff output, filters to risks with verdict "added" or "materially_changed", and issues one news search query per risk against the Tavily API. This step is a tool call because the LLM has no access to current events: asking it to produce news evidence would produce hallucinated citations.

Step 6 (LLM — Synthesizer + Critic) receives the entire accumulated state and produces two outputs in a single call: the formatted investor-ready markdown memo and a structured list of confidence assessments (High, Medium, or Low) for each material risk, based on an explicit rubric applied to the news evidence.

3. Tool Integration

Two external tools are integrated into the chain. The first is the SEC EDGAR public API, used in Step 2. EDGAR requires a `User-Agent` header identifying the caller but imposes no rate limit fee and requires no API key, making it accessible without registration. The tool fetches the filing submission index for a given CIK, identifies the two most recent 10-K forms, downloads their full HTML, and extracts the Item 1A section using regex anchors on the cleaned plain text. Its output enters the chain as a Python dictionary whose `latest_item_1a` and `prior_item_1a` keys are interpolated verbatim into the Step 3 user prompt.

The second tool is the Tavily search API, used in Step 5. Tavily was chosen over raw DuckDuckGo scraping as the primary source because it returns structured JSON with explicit publication dates, which the confidence rubric in Step 6 needs to distinguish sources from the last six months from older coverage. The fallback is transparent: it logs a warning to `state.errors` so the user sees it in the CLI summary and in the state JSON.

4. Limitations

The most consequential limitation is context truncation. Item 1A sections are truncated at 80,000 characters before being sent to Step 3. For companies with very large risk sections —

JPMorgan's Item 1A can exceed 120 pages — risks in the latter portion of the document are silently dropped. The agent has no mechanism to detect truncation or warn the user that coverage is incomplete, which means the diff may miss removals and additions that appear only in the truncated portion.

The regex-based Item 1A extractor in Step 2 is a second fragility point. It relies on the section heading "Item 1A" followed by "Risk Factors" appearing in the HTML-cleaned text. Some filers use inline tables, non-standard capitalisation, or embedded hyperlinks around section headings that cause the anchors to miss. In these cases the extractor logs a warning and falls back to the first 60,000 characters of the document, which may contain boilerplate before the risk section begins, degrading Step 3's extraction quality.

A related extraction failure was discovered during testing on large-cap companies: the original extractor used `re.search()`, which returns the first match of "Item 1A" in the document. Filings for companies such as MSFT and AMZN contain a table of contents near the top of the HTML document where "Item 1A" appears as a short navigation entry (~24 characters). The extractor matched this entry, found the next section boundary immediately after, and returned near-empty text — causing Step 3 to extract zero risks. The fix (collecting all matches and skipping any that produce fewer than 500 characters) is robust for the companies tested, but non-standard filing layouts could still produce matches where the real section is just barely above or below the threshold.

Foreign companies that list in the US but file 20-F forms rather than 10-K forms (Toyota, ASML, Shell, and most other non-US issuers) are not supported. The agent returns a clear error message when no 10-K filings are found for a given ticker, but it does not attempt to parse the 20-F format, which uses different section numbering (Item 3D rather than Item 1A).

5. Reflection

Given more time, the highest-priority change would be semantic chunking for large Item 1A sections. Rather than a hard character truncation, the section would be split into overlapping chunks of approximately 15,000 characters, risks would be extracted per chunk, and a deduplication pass would merge near-duplicate risks across chunks before the diff step. This would eliminate the silent coverage gap for large filers and is likely to be the single biggest accuracy improvement available.

The second priority would be 20-F support. The structural changes required are modest — a filing-type detector in Step 2 and a separate Item 3D extraction regex — but they would extend the agent's coverage to the majority of globally significant companies that are currently unsupported.

One non-obvious lesson from building this chain is that the label length constraint in Step 3 was far more impactful on downstream accuracy than any prompt engineering done in Step 4. The differ performs well once it receives short, atomic labels; it performs poorly regardless of how carefully the diff prompt is written if the labels are verbose. This suggests a general principle: investing prompt engineering effort at the extraction step (where raw data becomes structured data) yields larger downstream gains than investing it at the reasoning step (where

already-structured data is compared or synthesised). The v1-to-v2 iteration on the extraction prompt, documented in the Prompt Design Appendix, reduced spurious diff items on the TSLA test case from seventeen to zero — a more significant improvement than any version of the diff prompt achieved on its own.

Testing the agent against five real companies (MSFT, GOOGL, TSLA, AMZN, JPM) exposed three bugs that were not visible in unit tests. The table-of-contents extraction bug only appeared on large filers whose HTML documents contain a navigation section before the actual filing body — MSFT was the first company tested and immediately returned zero risks, revealing the issue. The rate-limit failure only appeared when running multiple companies in parallel, because Step 3 makes two sequential ~15k-token calls; a single-company run never approached the TPM limit. The Windows Unicode crash only appeared when a step failed during that parallel run, because the crash path (printing the failure symbol) was never exercised in earlier testing. This suggests that multi-company parallel execution should be part of any future test harness, as it creates conditions — rate pressure, failure paths, encoding edge cases — that single sequential runs do not.

An LLM (Claude) was used in a limited and specific capacity during development. The agent's domain (SEC 10-K risk delta analysis), the six-step chain architecture, the choice of tool calls at Steps 2 and 5, the Pydantic state design, and all prompt engineering decisions were conceived and designed independently. The LLM was used only for two low-level tasks: generating boilerplate code (initial Pydantic model field definitions, argparse setup, file I/O scaffolding) and assisting with debugging specific errors (the Windows Unicode crash and the regex match logic for the TOC bug). In both cases the generated output required correction — the Pydantic boilerplate omitted the `add_error()` helper and `to_json()` serialiser entirely, and the regex suggestion anchored on "Item 1A." with a mandatory period, missing the uppercase ITEM 1A variant used by several large filers. The chain design, step decomposition, prompt iteration strategy, and all architectural decisions throughout this project reflect independent reasoning, not LLM-generated suggestions.

See PROMPT_DESIGN.md for the full system and user prompts for each LLM step, including the Step 3 v1-to-v2 iteration with test-case evidence.